Indian Institute of Information Technology, Nagpur

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Distributed Computing System

Assignment 1

Under the Guidnace of

Dr. Milind Penurkar

Submitted by

Nimesh Maslekar BT21CSE004

Vikram Bhiwapurkar BT21CSE019

Varad Naik BT21CSE097

Distributed Calculator using .NET Remoting

1) Problem Statement

The purpose of this project is to build a distributed calculator application using .NET Remoting, where a client can interact with a remote server to perform basic mathematical operations. The operations include addition, subtraction, multiplication, and division. The system demonstrates how .NET Remoting can be used to enable communication between a client and a server over a network.

2) Methodology

The project is implemented using the following steps:

- **Step 1: Define the Shared Library:** A shared library (`SharedServices.dll`) is created containing the interface `ICalculator` which defines methods for basic mathematical operations. This interface acts as a contract between the client and the server, ensuring they both adhere to the same communication protocol.

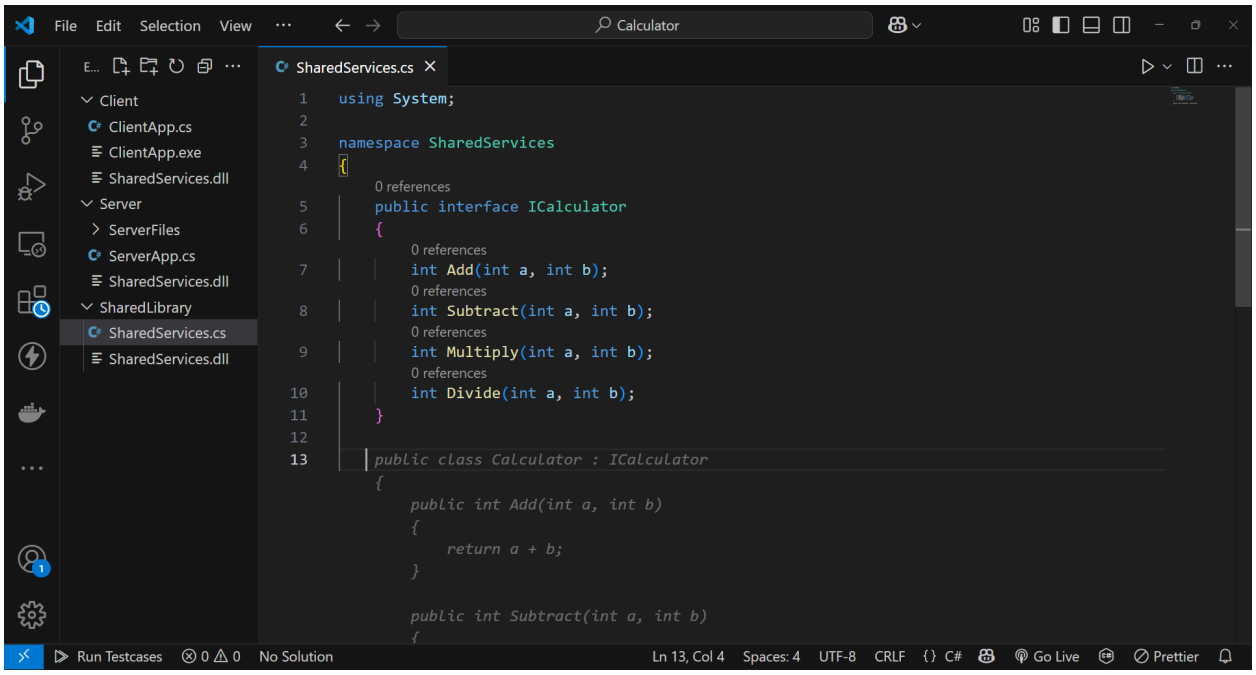
- **Step 2: Implement the Server:** The server application implements the `ICalculator` interface, providing the logic for performing calculations. The server registers the services using .NET Remoting and listens for client requests on a specified port. Thread safety mechanisms are implemented to handle concurrent requests if needed.

- **Step 3: Implement the Client:** The client application connects to the server and interacts with the calculator services. It provides a user interface for the user to select operations and supply operands. The client sends requests to the server and displays the results returned by the server.

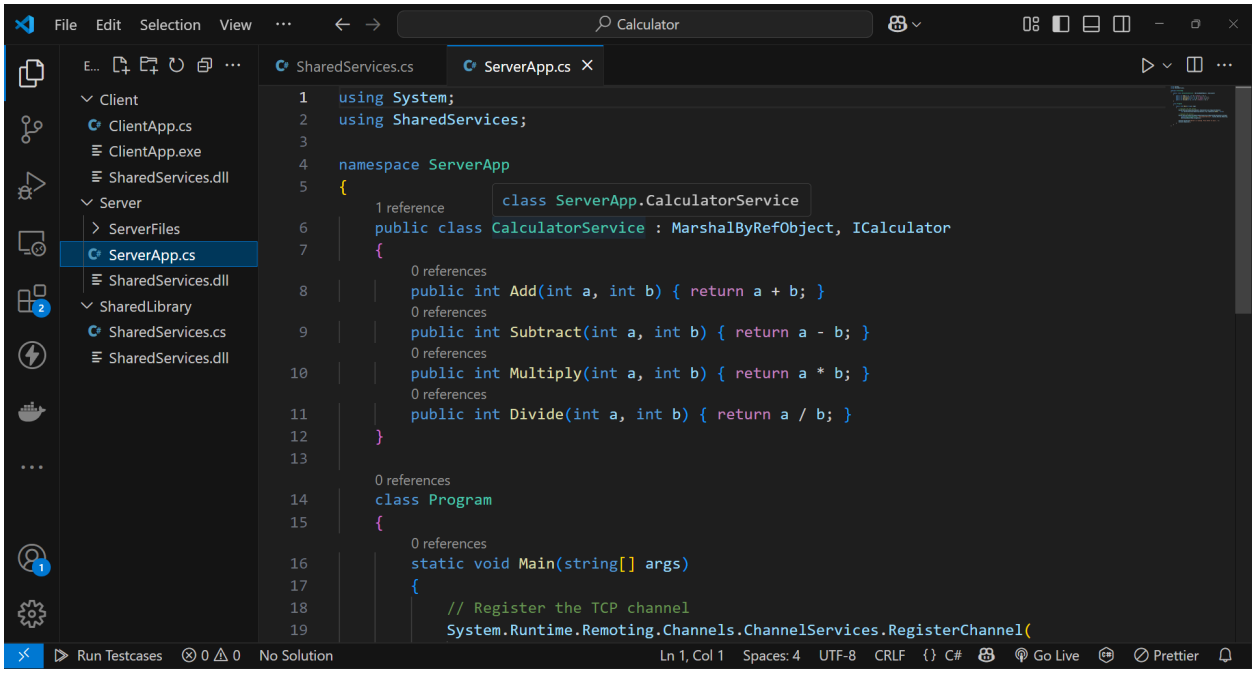
- **Step 4: Testing:** Testing involves ensuring all operations (addition, subtraction, multiplication, division) work correctly.

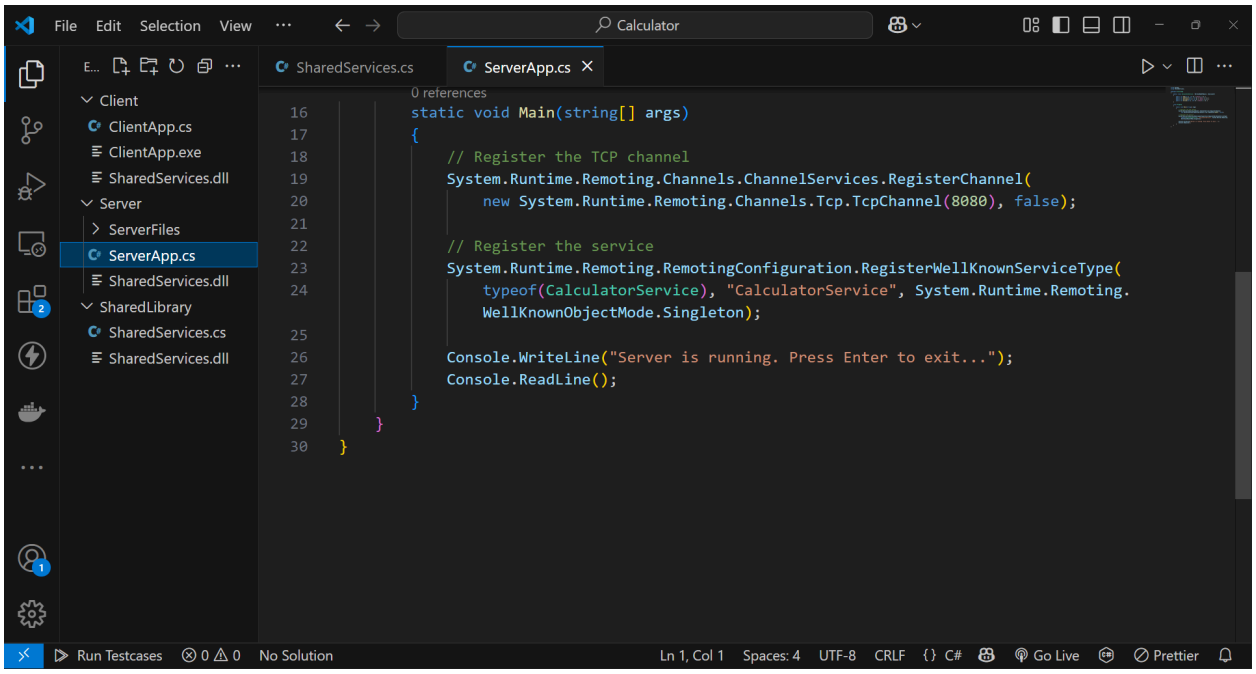
3) Code Snippets

Shared Library (SharedServices.cs):

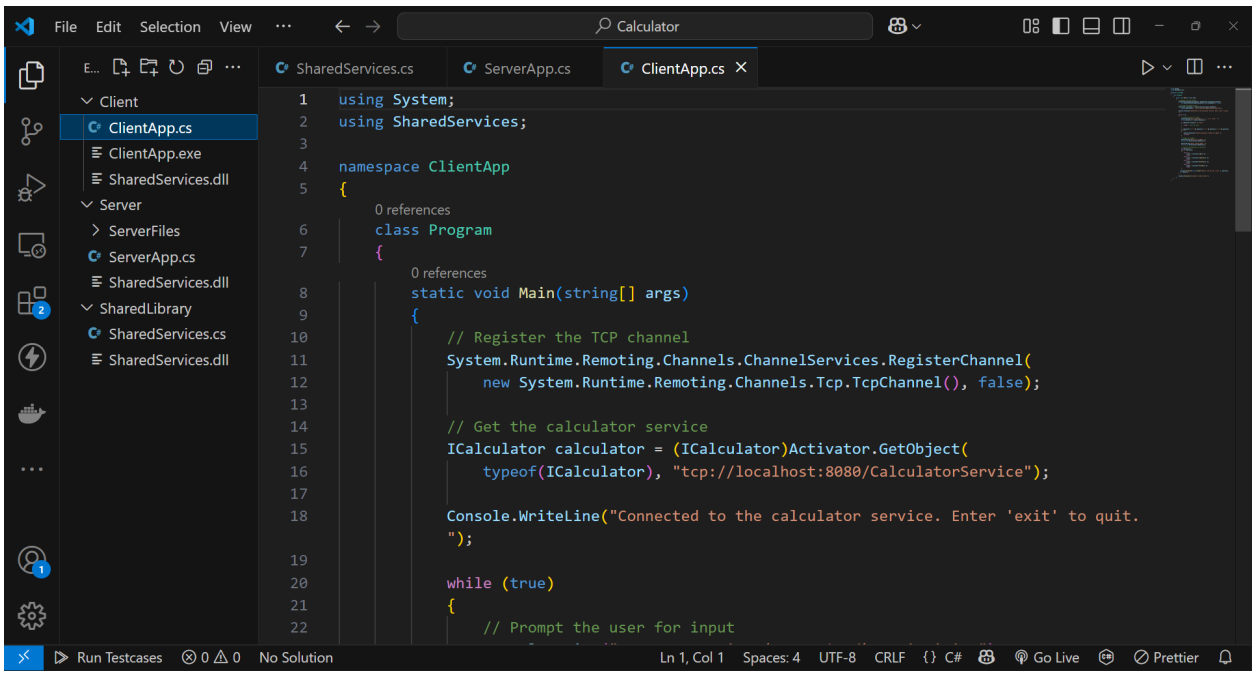


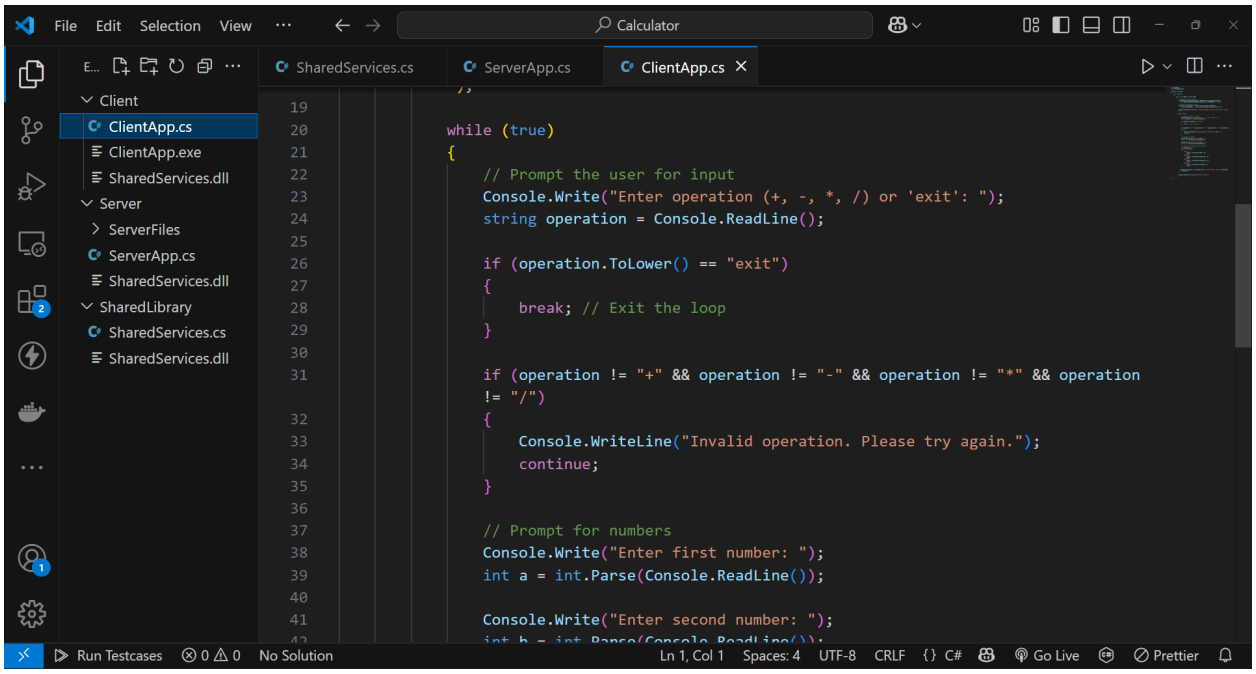
Server Implementation (ServerApp.cs):

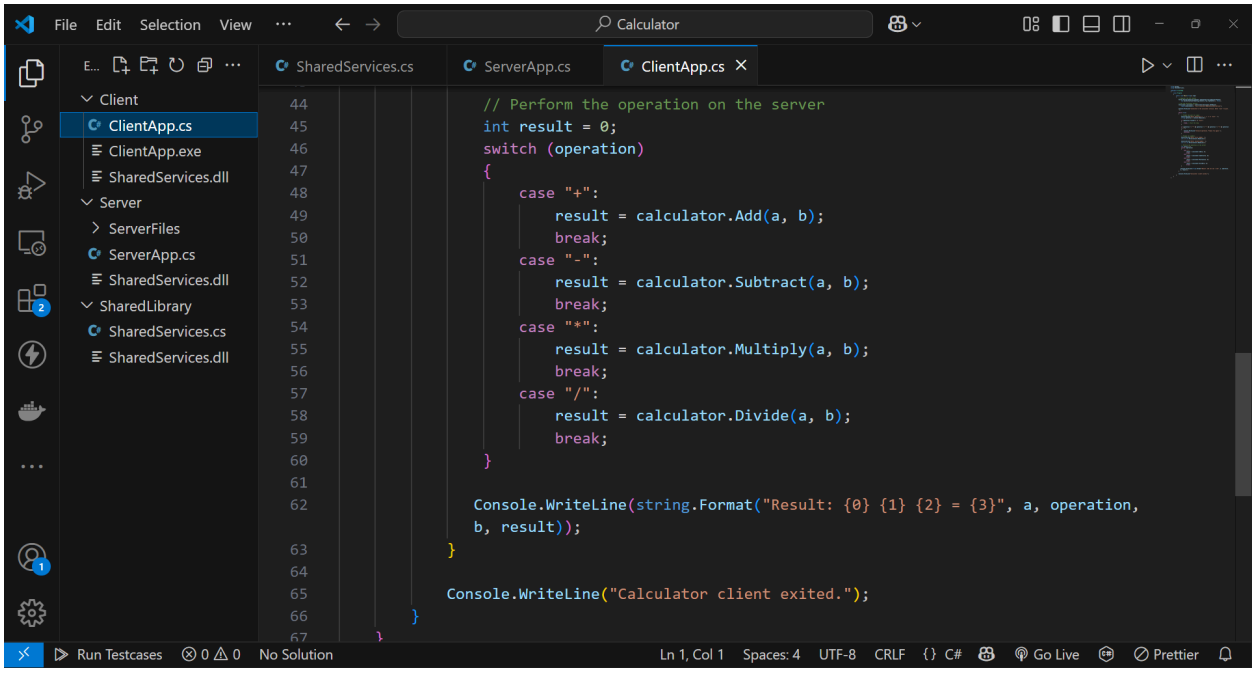




Client Implementation (ClientApp.cs):




File Edit Selection View
Calculator
SharedServices.cs
ServerApp.cs
ClientApp.cs
Client
ClientApp.cs
ClientApp.exe
SharedServices.dll
Server
ServerFiles
ServerApp.cs
SharedServices.dll
SharedLibrary
SharedServices.cs
SharedServices.dll
while (true)
{
// Prompt the user for input
Console.Write("Enter operation (+, -, *, /) or 'exit': ");
string operation = Console.ReadLine();
if (operation.ToLower() == "exit")
{
break; // Exit the loop
}
if (operation != "+" && operation != "-" && operation != "*" && operation
!= "/")
{
Console.WriteLine("Invalid operation. Please try again.");
continue;
}
// Prompt for numbers
Console.Write("Enter first number: ");
int a = int.Parse(Console.ReadLine());
Console.Write("Enter second number: ");
Run Testcases
0 0
No Solution
Ln 1, Col 1
Spaces: 4
UTF-8
CRLF
C#
Go Live
Prettier


File Edit Selection View
Calculator
SharedServices.cs
ServerApp.cs
ClientApp.cs
Client
ClientApp.cs
ClientApp.exe
SharedServices.dll
Server
ServerFiles
ServerApp.cs
SharedServices.dll
SharedLibrary
SharedServices.cs
SharedServices.dll
// Perform the operation on the server
int result = 0;
switch (operation)
{
case "+":
result = calculator.Add(a, b);
break;
case "-":
result = calculator.Subtract(a, b);
break;
case "*":
result = calculator.Multiply(a, b);
break;
case "/":
result = calculator.Divide(a, b);
break;
}
Console.WriteLine(string.Format("Result: {0} {1} {2} = {3}", a, operation,
b, result));
}
Console.WriteLine("Calculator client exited.");
}
Run Testcases
0 0
No Solution
Ln 1, Col 1
Spaces: 4
UTF-8
CRLF
C#
Go Live
Prettier

4) Method to Run the Project

1. **Compile the Shared Library:**
    - Compile `SharedServices.cs` to create `SharedServices.dll`

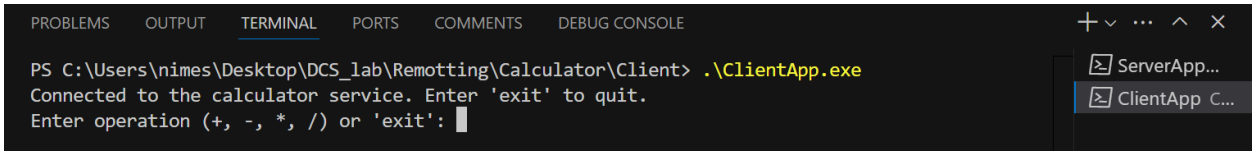
2. **Run the Server Application:**
    - Compile and run `ServerApp.cs`.
    - The server starts listening on a specified port (e.g., `tcp://localhost:8080`)

3. **Run the Client Application:**
    - Compile and run `ClientApp.cs`.
    - The client connects to the server and performs operations as requested by the user.

5) Outputs

Start the server

Start the client



Select the operation

```
Enter operation (+, -, *, /) or 'exit': +
Enter first number: 74
Enter second number: 45
Result: 74 + 45 = 119
Enter operation (+, -, *, /) or 'exit':
```

```
Enter operation (+, -, *, /) or 'exit': *
Enter first number: 4
Enter second number: 6
Result: 4 * 6 = 24
Enter operation (+, -, *, /) or 'exit':
```

```
Enter operation (+, -, *, /) or 'exit': -
Enter first number: 7
Enter second number: 5
Result: 7 - 5 = 2
```

```
Enter operation (+, -, *, /) or 'exit': /
Enter first number: 80
Enter second number: 5
Result: 80 / 5 = 16
Enter operation (+, -, *, /) or 'exit':
```

Exit the client and server

```
Enter operation (+, -, *, /) or 'exit': exit
Calculator client exited.
PS C:\Users\nimes\Desktop\DCS_lab\Remotting\Calculator\Client>
```

```
PS C:\Users\nimes\Desktop\DCS_lab\Remotting\Calculator\Server> .\ServerApp.exe
Server is running. Press Enter to exit...

PS C:\Users\nimes\Desktop\DCS_lab\Remotting\Calculator\Server>
```

Distributed File Transfer System Using .NET Remoting

1) Problem Statement

The goal of this project is to design and implement a distributed file transfer system using .NET Remoting. The system consists of:

- A server that hosts file transfer services.
- A client that interacts with the server to:
  - List files available on the server.
  - Download files from the server.
  - Upload files to the server

2) Methodology

The project is implemented using the following steps:

Step 1: Define the Shared Library

In this step, a shared library named `SharedServices.dll` is created to define the interfaces for the file transfer services. This library contains the `IFileTransfer` interface which specifies the essential methods for communication between the client and server. The interface includes methods for:

- **Listing files:** Retrieving a list of available files on the server.
- **Downloading files:** Allowing clients to request and download specific files from the server.
- **Uploading files:** Enabling clients to upload files to the server for storage.

The shared library serves as a contract between the client and server, ensuring both sides adhere to the same structure when interacting.

Step 2: Implement the Server

The server application is responsible for hosting the file transfer services defined in the shared library. The implementation of the `IFileTransfer` interface is carried out here. The server performs the following tasks:

- **Registering services:** The server registers the `IFileTransfer` services using .NET Remoting, making them accessible to clients over a network.
- **Listening for requests:** The server listens on a specified port for incoming client requests.

- **Handling file operations:** Provides functionality for listing, downloading, and uploading files.
- **Ensuring thread safety:** Implements mechanisms to handle concurrent requests from multiple clients effectively, preventing data corruption or other concurrency issues.

The server application is designed to run continuously and respond to client requests as they arrive.

Step 3: Implement the Client

The client application is developed to interact with the file transfer services hosted by the server. It uses the `IFileTransfer` interface defined in the shared library to communicate with the server. The client performs the following operations:

- **Connecting to the server:** Establishes a remote connection to the server using .NET Remoting.
- **Providing a user interface:** Presents a menu-driven interface allowing users to choose from available operations (listing, downloading, uploading files).
- **Requesting services:** Sends requests to the server and displays the received data or acknowledgment messages.
- **Handling errors:** Manages exceptions and errors related to network connectivity, file access, or other runtime issues.

The client is designed to be simple and user-friendly, ensuring smooth interaction with the server.

Step 4: Test the System

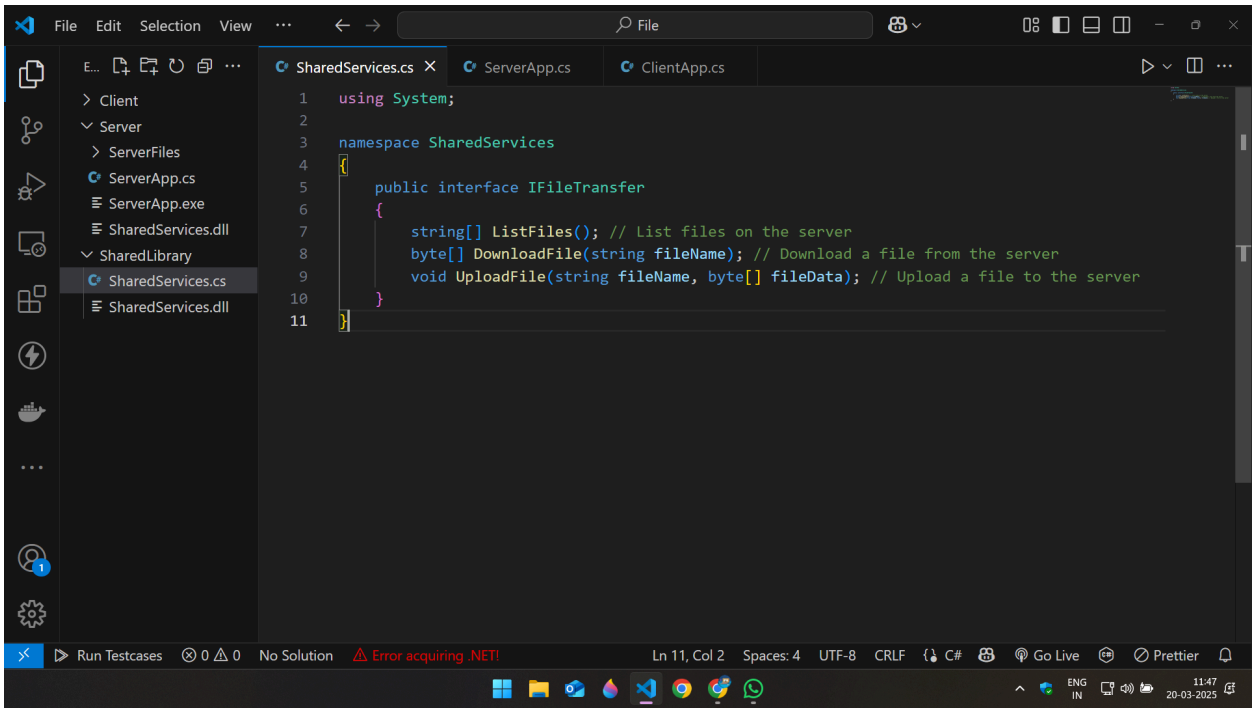
Comprehensive testing is performed to verify the functionality and robustness of the distributed file transfer system. This step involves:

- **Performing file operations:** Testing listing, downloading, and uploading files under various conditions.
- **Testing concurrency:** Ensuring thread safety by simulating concurrent requests from multiple clients to validate the server's ability to handle them without issues.
- **Validating data integrity:** Confirming that files are transferred accurately between the client and server.
- **Checking performance:** Measuring response times and system stability under load.

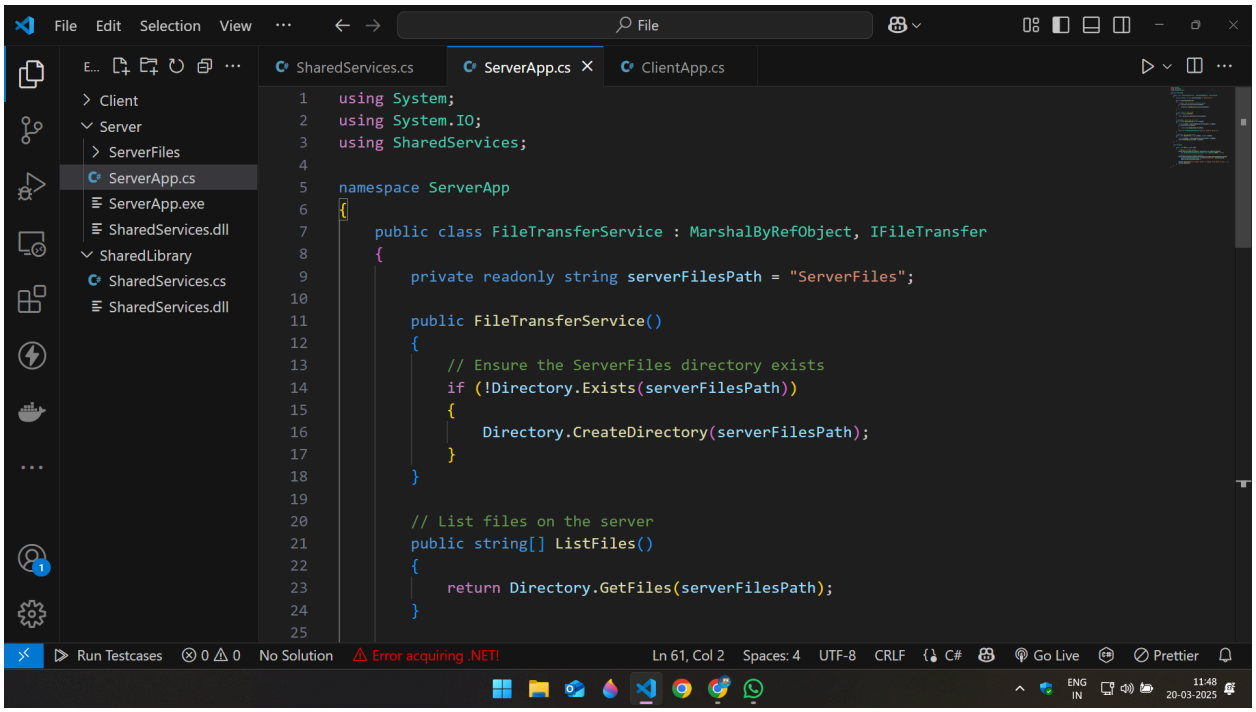
Testing ensures that the system meets all functional requirements and is capable of handling multiple client requests efficiently.

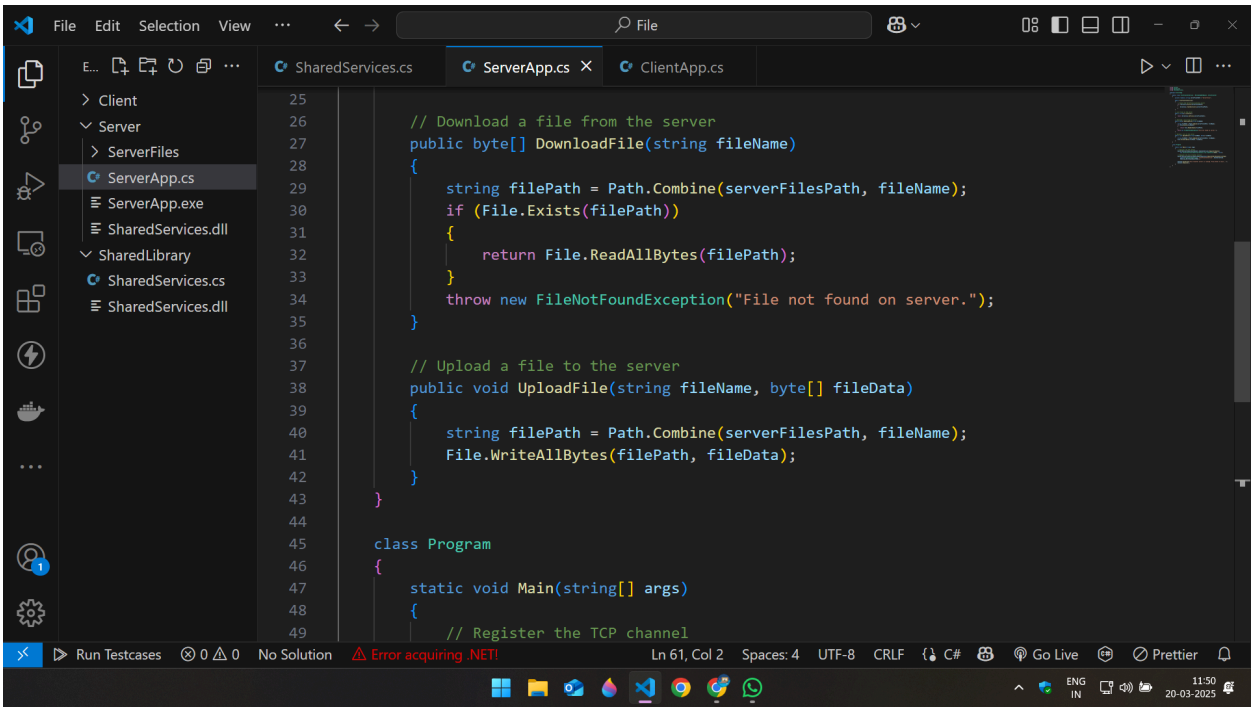
3) Code Snippets

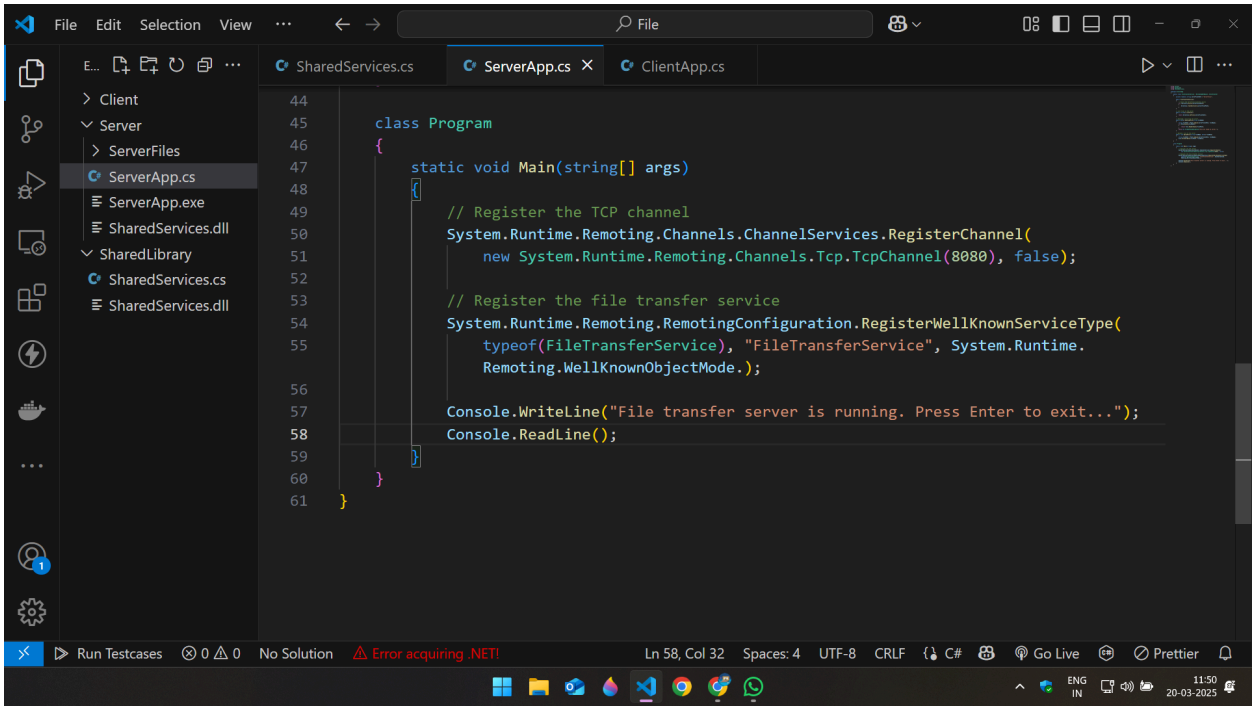
1. SharedServices.cs



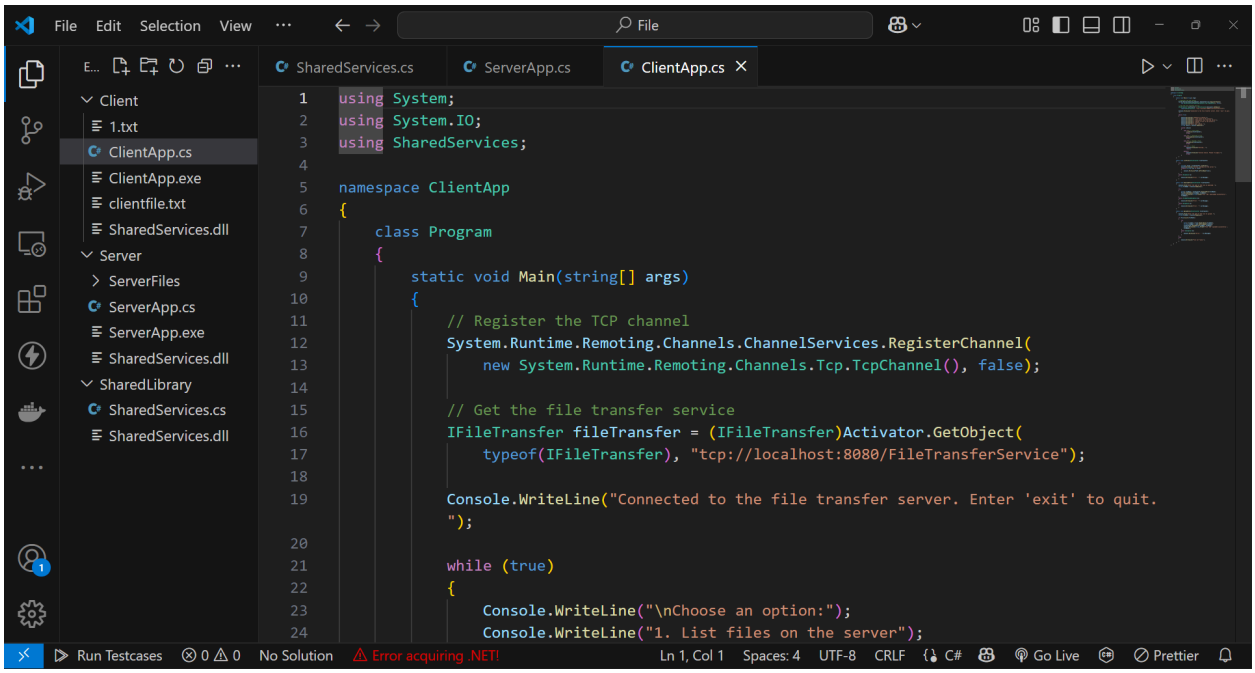
2. ServerApp.cs

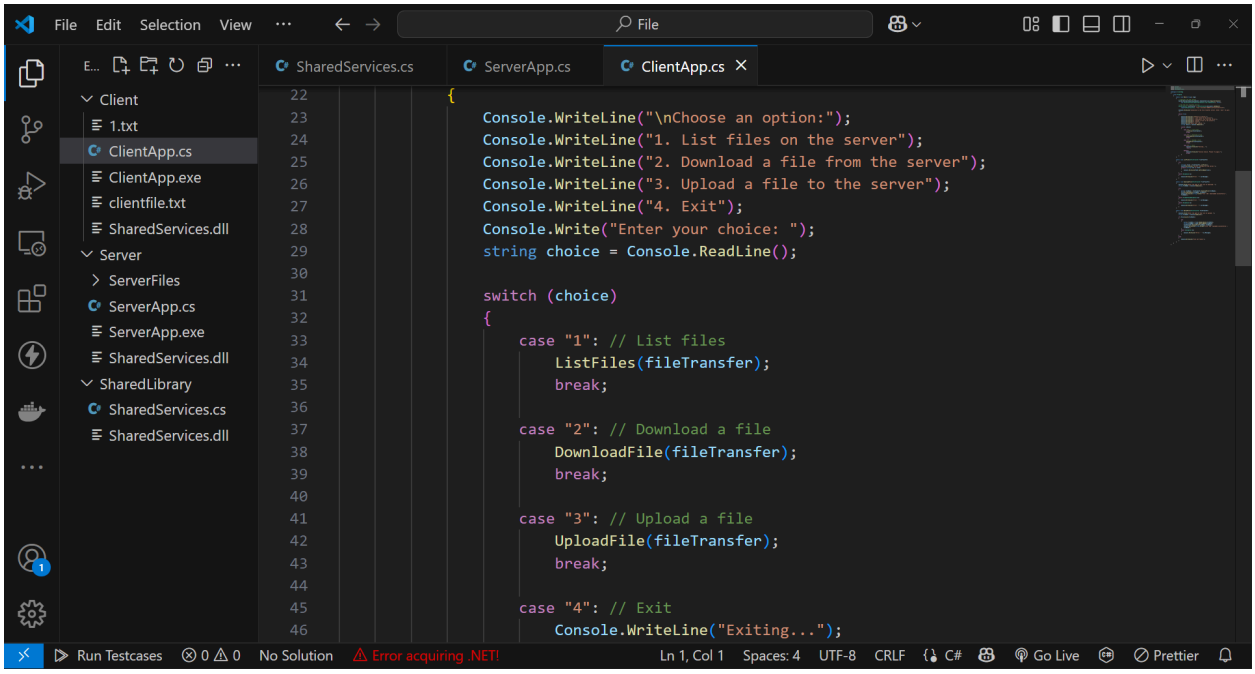


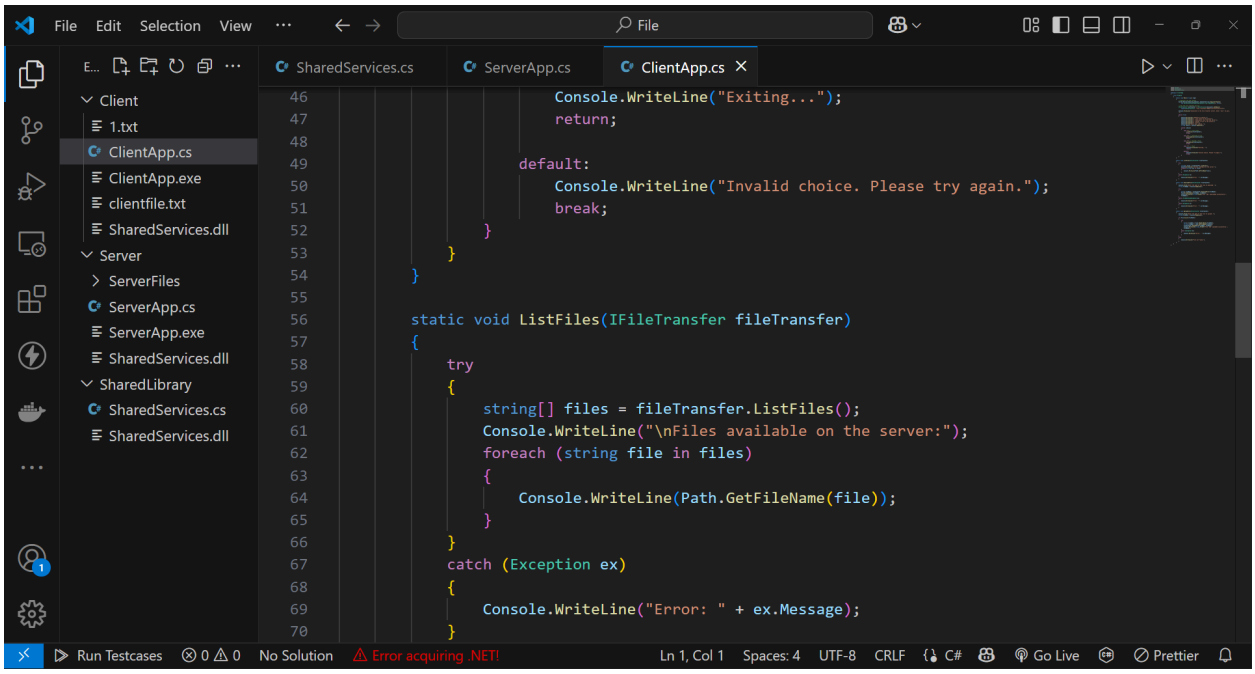

File Edit Selection View
SharedServices.cs
ServerApp.cs
ClientApp.cs
Client
Server
ServerFiles
ServerApp.cs
ServerApp.exe
SharedServices.dll
SharedLibrary
SharedServices.cs
SharedServices.dll
// Download a file from the server
public byte[] DownloadFile(string fileName)
{
string filePath = Path.Combine(serverFilesPath, fileName);
if (File.Exists(filePath))
{
return File.ReadAllBytes(filePath);
}
throw new FileNotFoundException("File not found on server.");
}
// Upload a file to the server
public void UploadFile(string fileName, byte[] fileData)
{
string filePath = Path.Combine(serverFilesPath, fileName);
File.WriteAllBytes(filePath, fileData);
}
}
class Program
{
static void Main(string[] args)
{
// Register the TCP channel
Run Testcases
No Solution
Error acquiring .NET!
Ln 61, Col 2
Spaces: 4
UTF-8
CRLF
C#
Go Live
Prettier

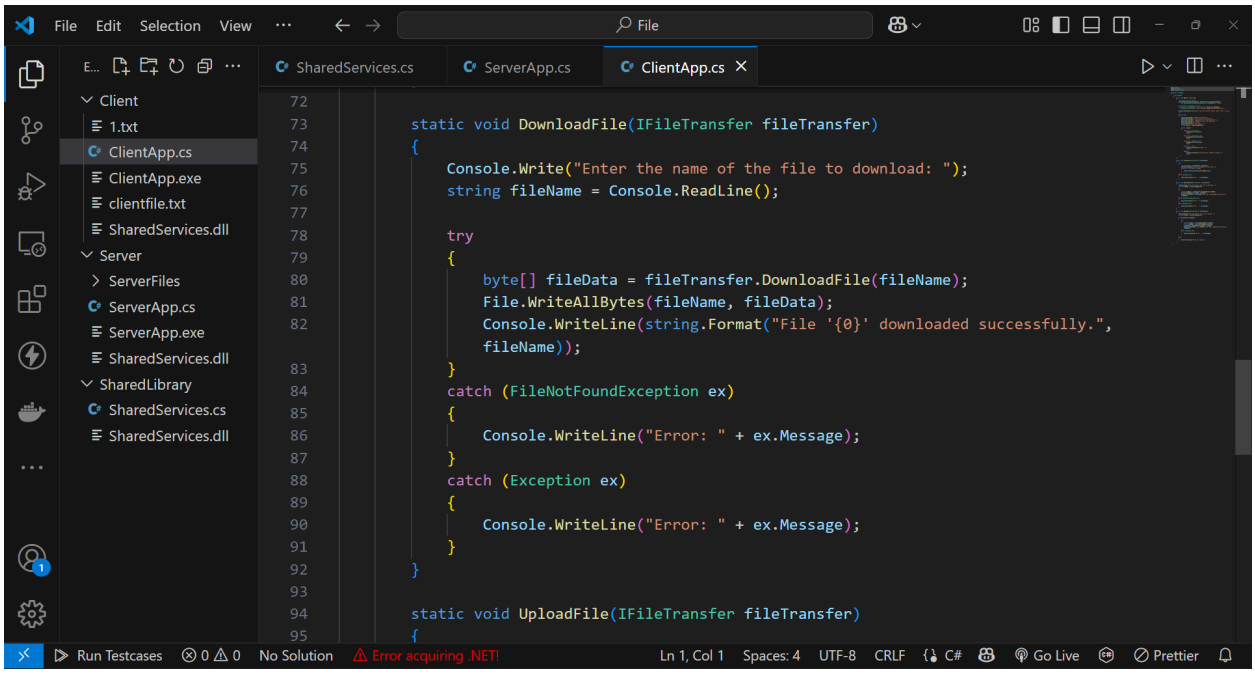

File Edit Selection View
SharedServices.cs
ServerApp.cs
ClientApp.cs
Client
Server
ServerFiles
ServerApp.cs
ServerApp.exe
SharedServices.dll
SharedLibrary
SharedServices.cs
SharedServices.dll
class Program
{
static void Main(string[] args)
{
// Register the TCP channel
System.Runtime.Remoting.Channels.ChannelServices.RegisterChannel(
new System.Runtime.Remoting.Channels.Tcp.TcpChannel(8080), false);
// Register the file transfer service
System.Runtime.Remoting.RemotingConfiguration.RegisterWellKnownServiceType(
typeof(FileTransferService), "FileTransferService", System.Runtime.
Remoting.WellKnownObjectMode.);
Console.WriteLine("File transfer server is running. Press Enter to exit...");
Console.ReadLine();
}
}
}
Run Testcases
No Solution
Error acquiring .NET!
Ln 58, Col 32
Spaces: 4
UTF-8
CRLF
C#
Go Live
Prettier

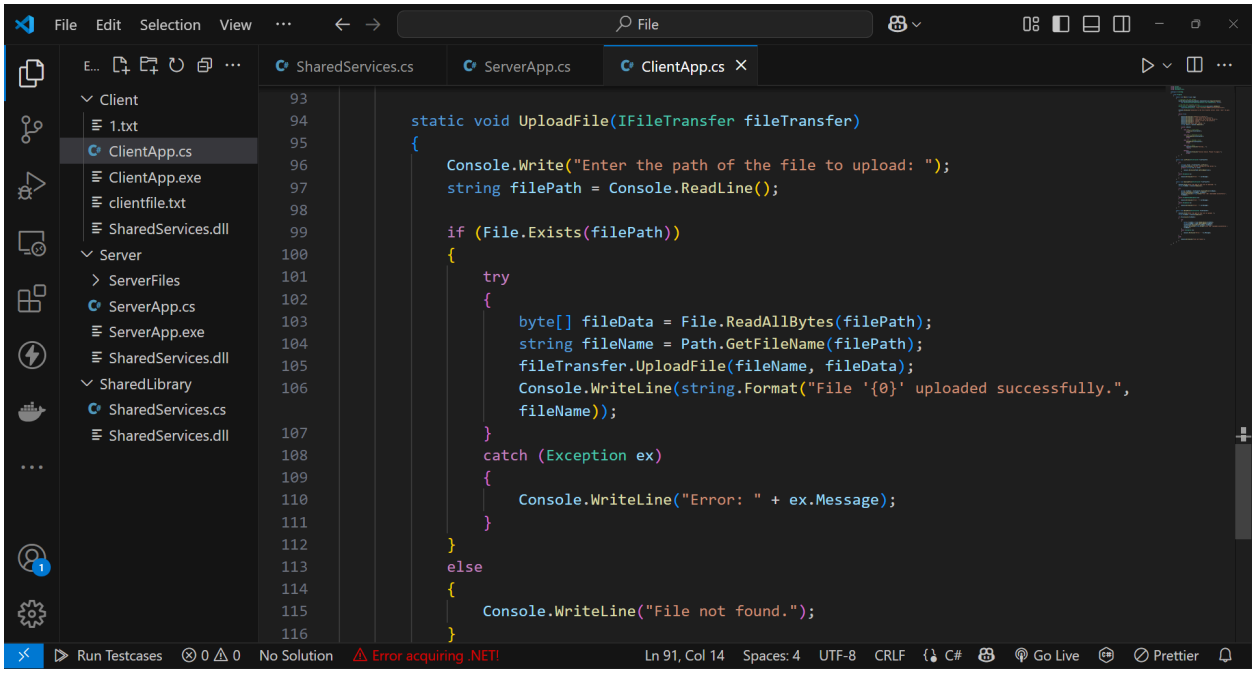
3. ClientApp.cs



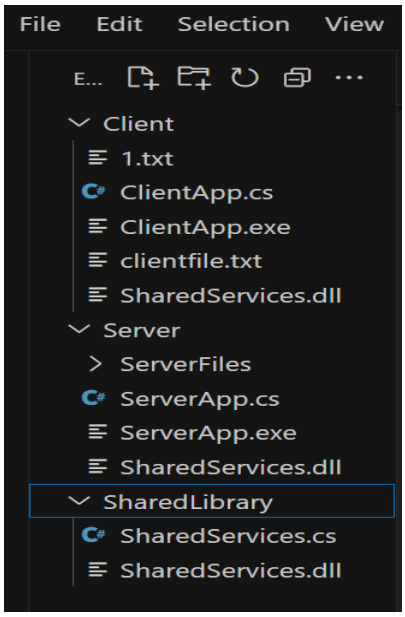



File Edit Selection View
File
SharedServices.cs
ServerApp.cs
ClientApp.cs
Client
1.txt
ClientApp.cs
ClientApp.exe
clientfile.txt
SharedServices.dll
Server
ServerFiles
ServerApp.cs
ServerApp.exe
SharedServices.dll
SharedLibrary
SharedServices.cs
SharedServices.dll
Console.WriteLine("Exiting...");
return;
default:
Console.WriteLine("Invalid choice. Please try again.");
break;
static void ListFiles(IFileTransfer fileTransfer)
try
string[] files = fileTransfer.ListFiles();
Console.WriteLine("\nFiles available on the server:");
foreach (string file in files)
Console.WriteLine(Path.GetFileName(file));
catch (Exception ex)
Console.WriteLine("Error: " + ex.Message);
Run Testcases
No Solution
Error acquiring .NET!
Ln 1, Col 1
Spaces: 4
UTF-8
CRLF
C#
Go Live
Prettier


File Edit Selection View
File
SharedServices.cs
ServerApp.cs
ClientApp.cs
Client
1.txt
ClientApp.cs
ClientApp.exe
clientfile.txt
SharedServices.dll
Server
ServerFiles
ServerApp.cs
ServerApp.exe
SharedServices.dll
SharedLibrary
SharedServices.cs
SharedServices.dll
static void DownloadFile(IFileTransfer fileTransfer)
Console.Write("Enter the name of the file to download: ");
string fileName = Console.ReadLine();
try
byte[] fileData = fileTransfer.DownloadFile(fileName);
File.WriteAllBytes(fileName, fileData);
Console.WriteLine(string.Format("File '{0}' downloaded successfully.",
fileName));
catch (FileNotFoundException ex)
Console.WriteLine("Error: " + ex.Message);
catch (Exception ex)
Console.WriteLine("Error: " + ex.Message);
static void UploadFile(IFileTransfer fileTransfer)
Run Testcases
No Solution
Error acquiring .NET!
Ln 1, Col 1
Spaces: 4
UTF-8
CRLF
C#
Go Live
Prettier



4. Directory Structure



4) Method to run the project

1. Compile the shared library
   ```
   csc /target:library SharedServices.cs
   ```

2. Compiler the server
   ```
   csc /reference:SharedServices.dll ServerApp.cs
   ```

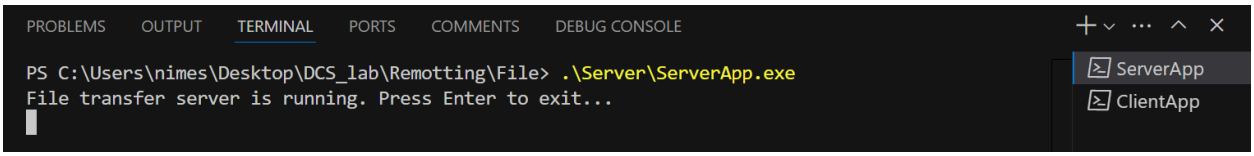
3. Compile the client
   ```
   csc /reference:SharedServices.dll ClientApp.cs
   ```

4. Run the server
   ```
   ServerApp.exe
   ```

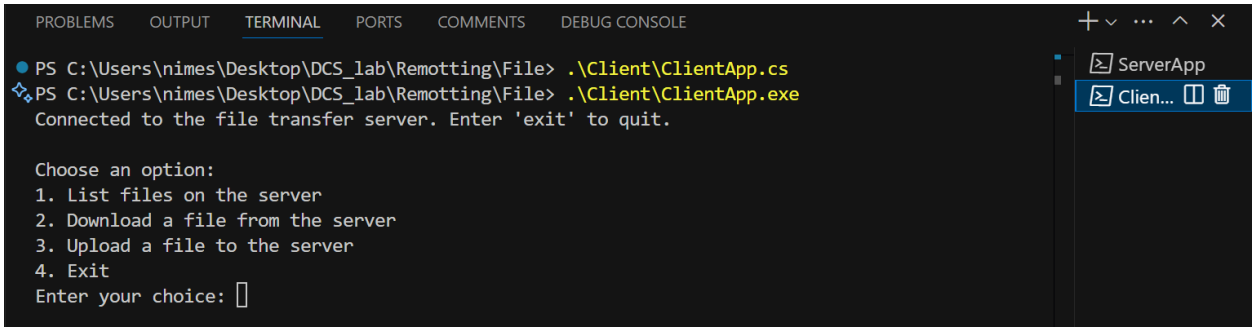
5. Run the client
   ```
   ClientApp.exe
   ```

5) Output

Start the server



Start the client



Choose a valid option

Download file from server

```
Enter your choice: 2
Enter the name of the file to download: serverfile.txt
File 'serverfile.txt' downloaded successfully.
```

Upload file to the server

```
4. Exit
Enter your choice: 3
Enter the path of the file to upload: to_be_sent_to_server.txt
File 'to_be_sent_to_server.txt' uploaded successfully.
```

Client exits

```
Choose an option:
1. List files on the server
2. Download a file from the server
3. Upload a file to the server
4. Exit
Enter your choice: 4
Exiting...
PS C:\Users\nimes\Desktop\DCS_lab\Remotting\File\Client>
```

Server exits